

4. Validació de formularis amb llibreries

En aquesta lliçó, explorarem com millorar la validació de formularis mitjançant l'ús de llibreries externes. Després d'una introducció a la creació d'una llibreria personalitzada per a validacions, aprendrem a utilitzar solucions més avançades com **JustValidate**, una llibreria lleugera i fàcil d'integrar. Aquesta lliçó inclou tant exemples pràctics com la creació d'un formulari complet amb validacions automàtiques, així com el treball amb eines modernes com Vite per millorar l'estructura i la gestió del projecte.

4.1 Programació d'una llibreria per validar formularis

Una **llibreria** és un conjunt de **funcions o mòduls reutilitzables** que resolen un problema concret.

En el cas dels formularis, una llibreria pot ajudar-nos a:

- Centralitzar totes les funcions de validació.
- Reduir codi repetit.
- Mantenir el codi més net i estructurat.
- Reutilitzar les mateixes regles en diferents pàgines o projectes.

A la lliçó 2 vam validar els formularis manualment, camp per camp. Ara crearem una llibreria que ho faci automàticament, passant-li només les regles i els camps.

4.1.1 Crear una llibreria bàsica de validació

Una llibreria no és més que un fitxer `.js` que conté funcions reutilitzables.

Comencem amb una estructura senzilla que encapsulem dins el fitxer `validacions.js`:

```
1 export function campObligatori(valor) {  
2   return valor.trim() !== "";  
3 }  
4  
5 export function longitudMinima(valor, mida) {  
6   return valor.trim().length >= mida;  
7 }  
8  
9 export function correuValid(valor) {  
10   const patron = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
11   return patron.test(valor);  
12 }
```

Aquestes funcions **retornen true o false** segons si el valor és correcte. Són senzilles però fàcilment ampliables.

Aquest fitxer l'hem creat pensant que serà utilitzat en un altre fitxer .js i és per això que hem exportat la seva funcionalitat, gràcies al fet que ja coneixem el treball amb mòduls.

4.1.2 Utilitzar la llibreria

Farem servir la llibreria validacions.js en el fitxer d'entrada de la nostra aplicació main.js:

```
1 import { campObligatori, longitudMinima, correuValid } from './
  validacions.js';
2
3 function validarFormulari() {
4   const nom = document.getElementById("nom").value;
5   const email = document.getElementById("email").value;
6   const missatge = document.getElementById("missatge");
7
8   if (!campObligatori(nom)) {
9     missatge.textContent = "El camp 'nom' és obligatori.";
10    return false;
11  }
12
13  if (!longitudMinima(nom, 3)) {
14    missatge.textContent = "El nom ha de tenir almenys 3 carà
      cters.";
15    return false;
16  }
17
18  if (!correuValid(email)) {
19    missatge.textContent = "El correu no té un format vàlid.";
20    return false;
21  }
22
23  missatge.textContent = "Formulari correcte!";
24  return true;
25 }
```

4.1.3 Connectar la lògica amb la vista HTML

script index.html:

```
1 <!DOCTYPE html>
2 <html lang="ca">
3 <head>
4   <meta charset="UTF-8">
5   <title>Validació amb llibreria pròpia</title>
6 </head>
7 <body>
8
```

```
9 <h3>Formulari de contacte</h3>
10 <form id="formulari" onsubmit="return validarFormulari();">
11   <label for="nom">Nom:</label>
12   <input type="text" id="nom">
13
14   <label for="email">Correu electrònic:</label>
15   <input type="email" id="email">
16
17   <button type="submit">Enviar</button>
18   <p id="missatge"></p>
19 </form>
20
21 <script type="module" src="main.js"></script>
22 </body>
23 </html>
```

Fixem-nos que:

- Estem treballant amb mòduls ES6.
- El formulari no s'enviarà mai perquè dins de main.js aturem el comportament per defecte amb `event.preventDefault();`. Si volguéssim enviar el formulari després de les validacions senzillament hauríem de canviar la línia `missatge.textContent = "Formulari correcte!"`; per `document.getElementById("formulari").submit();`.

Ja podem testejar el nostre formulari, i com veiem en la figura 5.1 i la figura 5.2 tenim la funcionalitat ben implementada.

També vam treballar amb mòduls ES6 a la lliçó 3.

FIGURA 4.1. Exemple del codi de validació del formulari amb llibreries en el cas que falti algun camp

The screenshot shows a web browser window with the title "Validació amb llibreria pròpia". The address bar shows the URL "127.0.0.1:5500/index.html". The page content includes a heading "Formulari de contacte". Below the heading, there is a form with two input fields: "Nom:" followed by a text input containing "Joan", and "Correu electrònic:" followed by an empty email input field. To the right of the email input is an "Enviar" button. Below the form, a message states "El correu no té un format vàlid."

FIGURA 4.2. Exemple del codi de validació del formulari amb llibreries en el cas que la validació sigui correcta



4.1.4 Afegir missatges personalitzats i estil visual

Podem millorar tot el codi anterior per tal que els missatges d'error ens sortissin just sota l'element equivocat amb un estil visual adequat per millorar l'experiència de l'usuari.

Al fitxer `index.html` definirem uns estils i crearem unes etiquetes ocultes amb els **missatges d'error**:

```

1 <!DOCTYPE html>
2 <html lang="ca">
3   <head>
4     <meta charset="UTF-8" />
5     <title>Formulari amb validació (mòduls)</title>
6     <meta name="viewport" content="width=device-width, initial-
7       scale=1" />
8     <style>
9       form { max-width: 420px; margin: 2rem auto; font-family:
10         system-ui, sans-serif; }
11       label { display: block; margin-top: 1rem; font-weight:
12         600; }
13       input { width: 100%; padding: .6rem; margin-top: .35rem;
14         border: 1px solid #bbb; border-radius: .5rem; }
15       button { margin-top: 1rem; padding: .7rem 1rem; border: 0;
16         border-radius: .6rem; cursor: pointer; }
17       .error { display: block; margin-top: .25rem; }
18     </style>
19   </head>
20   <body>
21     <h3 style="text-align:center">Formulari de contacte</h3>
22
23     <form id="formulari">
24       <div>
25         <label for="nom">Nom</label>
26         <input type="text" id="nom" name="nom" />
27         <span
28           class="error"

```

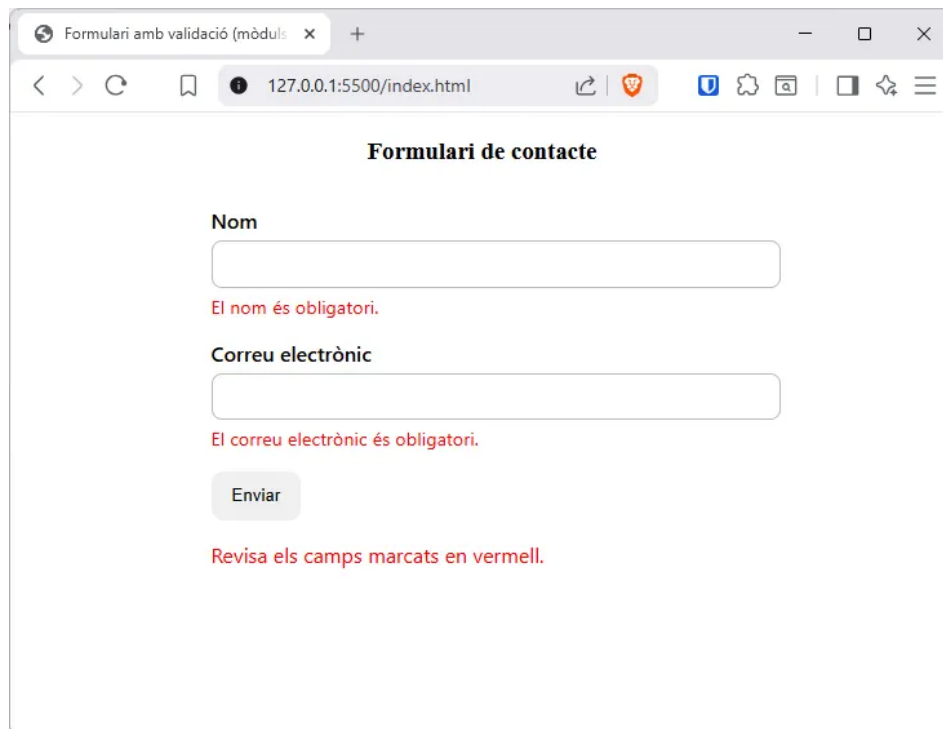
```
24         style="color: red; font-size: 0.9rem; display: none;"
25         >El nom és obligatori.</span>
26     >
27 </div>
28
29 <div>
30     <label for="email">Correu electrònic</label>
31     <input type="email" id="email" name="email" />
32     <span
33         class="error"
34         style="color: red; font-size: 0.9rem; display: none;"
35         >El correu electrònic és obligatori.</span>
36     >
37 </div>
38
39 <button type="submit">Enviar</button>
40 <p id="missatge"></p>
41 </form>
42
43 <script type="module" src="./main.js"></script>
44 </body>
45 </html>
```

I ampliarem l'arxiu main.js afegint els canvis:

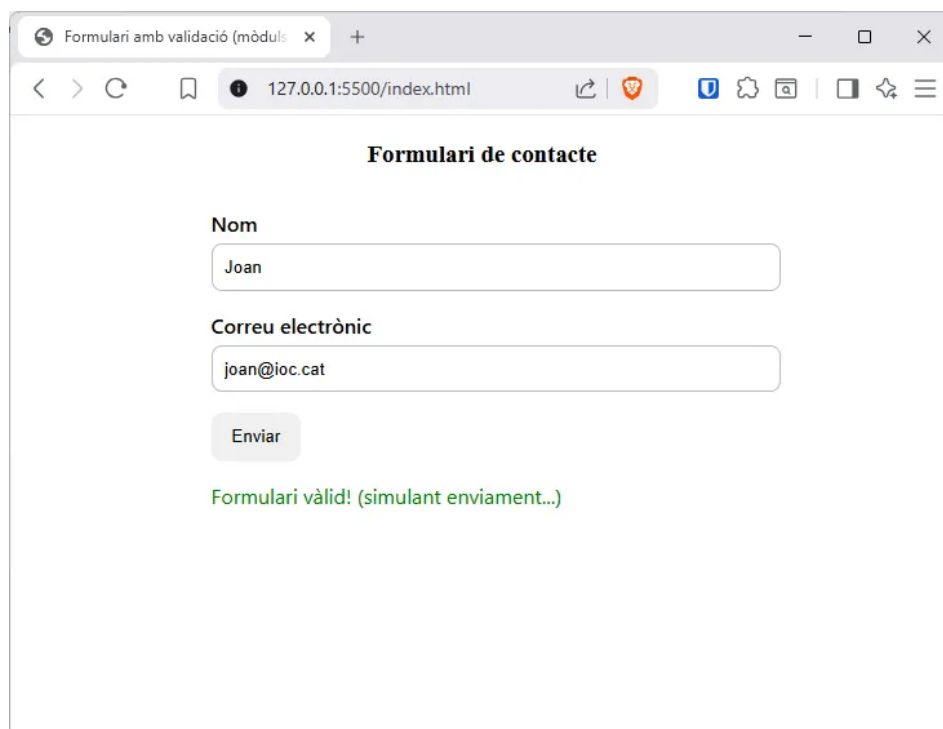
```
1  import { campObligatori, longitudMinima, correuValid } from './
   validacions.js';
2
3  function mostraError(input) {
4      let span = input.parentElement.querySelector('.error');
5      span.style.display = 'block';
6  }
7
8  function netejaError(input) {
9      const span = input.parentElement.querySelector('.error');
10     span.style.display = 'none';
11 }
12
13 function validaCampNom(input) {
14     const valor = input.value;
15     if (!campObligatori(valor)) {
16         mostraError(input);
17         return false;
18     }
19     if (!longitudMinima(valor, 2)) {
20         mostraError(input);
21         return false;
22     }
23     netejaError(input);
24     return true;
25 }
26
27 function validaCampEmail(input) {
28     const valor = input.value;
29     if (!campObligatori(valor)) {
30         mostraError(input, 'El correu és obligatori.');
```

```
36   }
37   netejaError(input);
38   return true;
39 }
40
41 const form = document.getElementById('formulari');
42 const nom = document.getElementById('nom');
43 const email = document.getElementById('email');
44 const missatge = document.getElementById('missatge');
45
46 // Validació en esdeveniments de sortida/canvi per millorar UX
47 // Quan l'usuari surt del camp, el validem
48 nom.addEventListener('blur', () => validaCampNom(nom));
49 email.addEventListener('blur', () => validaCampEmail(email));
50
51 form.addEventListener('submit', (e) => {
52   e.preventDefault(); // evita l'enviament automàtic
53
54   const nomOK = validaCampNom(nom);
55   const emailOK = validaCampEmail(email);
56
57   if (nomOK && emailOK) {
58     missatge.textContent = 'Formulari vàlid! (simulant
59       enviament...)';
60     missatge.style.color = 'green';
61
62     nom.netejaError();
63     email.netejaError();
64   } else {
65     missatge.textContent = 'Revisa els camps marcats en
66       vermell.';
67     missatge.style.color = 'red';
68   }
69 });
```

La nostra llibreria `validacions.js` com que no té cap relació amb la vista no pateix cap canvi respecte a la versió del punt anterior. El comportament del formulari després d'afegir la capa d'estils el podem veure en la figura 5.3 i la figura 5.4.

FIGURA 4.3. Formulari amb camps obligatoris. Missatge d'error

The screenshot shows a web browser window with the title "Formulari amb validació (mòdul: x)". The address bar shows "127.0.0.1:5500/index.html". The page content is titled "Formulari de contacte". It contains two input fields: "Nom" and "Correu electrònic". Below the "Nom" field, there is a red error message: "El nom és obligatori." Below the "Correu electrònic" field, there is a red error message: "El correu electrònic és obligatori." Below these fields is a button labeled "Enviar". At the bottom of the form, there is a red message: "Revisa els camps marcats en vermell."

FIGURA 4.4. Es compleixen les regles de validació

The screenshot shows the same web browser window as Figure 4.3. The "Nom" field now contains the text "Joan" and the "Correu electrònic" field contains the text "joan@ioc.cat". The "Enviar" button is still present. Below the button, there is a green message: "Formulari vàlid! (simulant enviament...)"

Fixem-nos que per aconseguir aquest estil visual hem hagut de manipular manualment el DOM, mostrant i ocultant etiquetes, que és quelcom que estudiarem amb més calma més endavant.

Aquesta estructura és **molt flexible i pot créixer** fins a convertir-se en una llibreria completa de validació per reutilitzar-la en altres projectes.

4.2 Llibreries per validar formularis

Quan el projecte creix, escriure totes les validacions manualment pot resultar feixuc.

Per això, hi ha **llibreries ja preparades** que ens ajuden a validar formularis de manera més ràpida i consistent. A la taula 4.1 podeu veure algunes de les més conegudes:

TAULA 4.1. Llibreries de validació de formularis

Llibreria	Característiques principals
JustValidate	Lleugera, sense dependències, molt fàcil d'integrar en projectes moderns.
Parsley.js	Proporciona validació declarativa amb atributs HTML i missatges personalitzables.
Validator.js	Centrada en validar valors (emails, telèfons, dates, números, etc.) dins del JS, no directament al DOM.
Yup	Sovint usada amb React i <i>frameworks</i> moderns per validar objectes complexos.
Formik + Yup	Combinació molt utilitzada per gestionar i validar formularis en React.

4.2.1 Exemple pràctic amb JustValidate

Escollim **JustValidate** com a llibreria de validació de formularis, i amb ella farem un exemple pràctic. El procés serà el següent:

1. Afegir la llibreria
2. Crear el formulari
3. Configurar la validació
4. Resultat final

Afegir la llibreria

Podem importar **JustValidate** des de CDN al nostre HTML:

```
1 <script src="https://cdn.jsdelivr.net/npm/just-validate@latest/dist/just-validate.production.min.js"></script>
```

Un **CDN** (*Content Delivery Network*) és una xarxa de servidors distribuïts geogràficament que permet que els fitxers es carreguin més ràpidament des del servidor més proper a la ubicació de l'usuari.

Crear el formulari

```
1 <form id="formulariContacte">
2   <div class="form-group">
3     <label for="nom">Nom complet:</label>
4     <input type="text" id="nom" name="nom" />
5   </div>
6   <div class="form-group">
7     <label for="email">Correu electrònic:</label>
8     <input type="email" id="email" name="email" />
9   </div>
10  <div class="form-group">
11    <label for="edat">Edat:</label>
12    <input type="number" id="edat" name="edat" />
13  </div>
14  <button type="submit">Enviar</button>
15 </form>
16
17 <p id="missatge"></p>
```

Configurar la validació

Quan usem una llibreria com **JustValidate**, no cal que nosaltres escrivim manualment totes les funcions per comprovar si un camp està buit, si té prou caràcters o si un correu té el format correcte.

La llibreria ja ens ofereix **regles predefinides** que podem aplicar fàcilment a cada camp del formulari.

Aquí tenim el codi:

```
1 <script>
2   const validator = new JustValidate('#formulariContacte');
3
4   validator
5     .addField('#nom', [
6       { rule: 'required', errorMessage: 'El nom és obligatori' },
7       { rule: 'minLength', value: 3, errorMessage: 'Mínim 3 caràcters' },
8     ])
9     .addField('#email', [
10      { rule: 'required', errorMessage: 'El correu és obligatori' },
11      { rule: 'email', errorMessage: 'El correu no és vàlid' },
12    ])
13     .addField('#edat', [
14      { rule: 'required', errorMessage: 'L'edat és obligatòria' },
15    ])
```

```
15     { rule: 'number', errorMessage: 'Només es permeten números  
16         ' },  
17     { rule: 'minNumber', value: 18, errorMessage: 'Has de ser  
18         major d'edat' },  
19     ])  
20     .onSuccess(() => {  
21         document.getElementById("missatge").textContent = "  
    Formulari correcte!";  
    });  
</script>
```

Abans d'escriure el codi anterior, ens hem hagut de preguntar:

1. Quin formulari volem validar?

Volem validar al formulari amb *id* `formulariContacte`. Això ho aconseguim creat l'objecte:

```
1 const validator = new JustValidate('#formulariContacte');
```

2. Quins camps del formulari volem comprovar?

- Nom
- Email
- Edat

3. Quines regles s'han d'aplicar a cada camp?

- Pel camp **nom** volem que:
 - Existeixi.
 - Tingui una longitud mínima de 3 caràcters.
- Pel camp **email** volem que:
 - Existeixi.
 - Tingui el format que **JustValidate** consideri que ha de tenir una adreça de correu electrònic.
- Pel camp **edat** volem que:
 - Existeixi.
 - Sigui de tipus numèric.
 - El valor sigui més gran que 18.

4. Quins missatges d'error s'han de mostrar si no es compleix una regla?

Cada regla té l'atribut `errorMessage` on definim el missatge en cas que no es compleixi la regla.

Aquesta llibreria detecta automàticament els errors i mostra els missatges al costat dels camps sense necessitat d'escriure gaire codi.

Resultat final

Si posem tot el codi anterior en un sol fitxer .html i afegim una mica d'estil tenim el següent codi:

```
1 <!DOCTYPE html>
2 <html lang="ca">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-
      scale=1.0" />
6     <title>Validació de Formulari amb JustValidate</title>
7     <!-- Importar JustValidate des de CDN -->
8     <script src="https://cdn.jsdelivr.net/npm/just-
      validate@latest/dist/just-validate.production.min.js"></
      script>
9   <style>
10    /* Estils bàsics per al formulari i els missatges d'error
      */
11    body {
12      font-family: Arial, sans-serif;
13      margin: 20px;
14    }
15    form {
16      max-width: 400px;
17      padding: 20px;
18      border: 1px solid #ccc;
19      border-radius: 8px;
20    }
21    .form-group {
22      margin-bottom: 15px;
23    }
24    label {
25      display: block;
26      margin-bottom: 5px;
27      font-weight: bold;
28    }
29    input[type="text"],
30    input[type="number"],
31    input[type="email"] {
32      width: 100%;
33      padding: 10px;
34      border: 1px solid #ccc;
35      border-radius: 4px;
36      box-sizing: border-box; /* Important per incloure
      padding al width */
37    }
38    button {
39      background-color: #4CAF50;
40      color: white;
41      padding: 10px 15px;
42      border: none;
43      border-radius: 4px;
44      cursor: pointer;
45    }
46    /* Estil per als missatges d'error de JustValidate */
47    .js-validate-error-label {
48      color: red;
49      font-size: 0.9em;
50      margin-top: 5px;
51    }
52  </style>
```

```
53 </head>
54 <body>
55   <div class="container">
56     <h3>Formulari de Contacte</h3>
57     <form id="formulariContacte">
58       <div class="form-group">
59         <label for="nom">Nom complet:</label>
60         <input type="text" id="nom" name="nom" />
61       </div>
62       <div class="form-group">
63         <label for="email">Correu electrònic:</label>
64         <input type="email" id="email" name="email" />
65       </div>
66       <div class="form-group">
67         <label for="edat">Edat:</label>
68         <input type="number" id="edat" name="edat" />
69       </div>
70       <button type="submit">Enviar</button>
71     </form>
72     <p id="missatge"></p>
73   </div>
74
75   <script>
76     // Inicialitzar JustValidate
77     const validator = new JustValidate('#formulariContacte', {
78       // Configuració opcional per localitzar o canviar l'
79       // estil
80       errorFieldCssClass: 'is-invalid',
81       errorLabelCssClass: 'js-validate-error-label',
82     });
83
84     // Afegir regles de validació per als camps
85     validator
86       .addField('#nom', [
87         { rule: 'required', errorMessage: 'El nom és
88           obligatori' },
89         { rule: 'minLength', value: 3, errorMessage: 'Mínim 3
90           caràcters' },
91       ])
92       .addField('#email', [
93         { rule: 'required', errorMessage: 'El correu és
94           obligatori' },
95         { rule: 'email', errorMessage: 'El correu no és vàlid'
96           },
97       ])
98       .addField('#edat', [
99         { rule: 'required', errorMessage: "L'edat és obligatò
100           ria" },
101         { rule: 'number', errorMessage: 'Només es permeten nú
102           meros' },
103         { rule: 'minNumber', value: 18, errorMessage: "Has de
104           ser major d'edat" },
105       ])
106       .onSuccess(() => {
107         // Mostrar missatge d'èxit quan totes les validacions
108         // són correctes
109         document.getElementById("missatge").textContent = "
110           Formulari correcte!";
111         document.getElementById("missatge").className = "
112           success-message";
113       });
114   </script>
115 </body>
```

```
105 </html>
```

Ens hem de fixar que hem afegit algun detall més al constructor de l'objecte validator per tal que ens mostri els errors sota el camp input que conté l'error:

```
1 const validator = new JustValidate("#formulariContacte", {  
2   // Configuració opcional per localitzar o canviar l'estil  
3   errorFieldCssClass: "is-invalid",  
4   errorLabelCssClass: "js-validate-error-label",  
5 });
```

I amb el codi anterior aconseguim un formulari semblant al de la figura 5.5.

FIGURA 4.5. Formulari de contacte amb camps obligatoris

Llibreria JustValidate

Podem trobar la documentació de la llibreria JustValidate a just-validate.dev.

Formulari de Contacte

El formulari de contacte està presentat dins d'un container amb una vora arrodonida i un fons gris clar. Conté tres camps d'entrada, cadascun amb un label a l'esquerra i un missatge d'error en vermell a sota si no s'ha omplert. Els camps són: 'Nom complet:', 'Correu electrònic:' i 'Edat:'. A la part inferior esquerra hi ha un botó verd amb el text 'Enviar'.

Nom complet:

El nom és obligatori

Correu electrònic:

El correu és obligatori

Edat:

L'edat és obligatòria

Enviar

4.2.2 Treball modern amb JustValidate

Ara reescriurem l'exemple del punt anterior, però des d'una aproximació més moderna, aprofitant l'aprenentatge d'ús d'empaquetadors de la lliçó anterior. El procés serà el següent:

1. Crear un nou projecte amb Vite
2. Instal·lació de JustValidate
3. Creació del formulari i les regles de validació

4. Execució del projecte

Crear un nou projecte amb Vite

Crearem un nou projecte amb **Vite** i utilitzem la plantilla **vanilla**:

```
1 npm create vite@latest projecte-justvalidate
```

Si no ens ho fa sol, llençarem `npm install` per tenir les dependències instal·lades.

Instal·lació de JustValidate

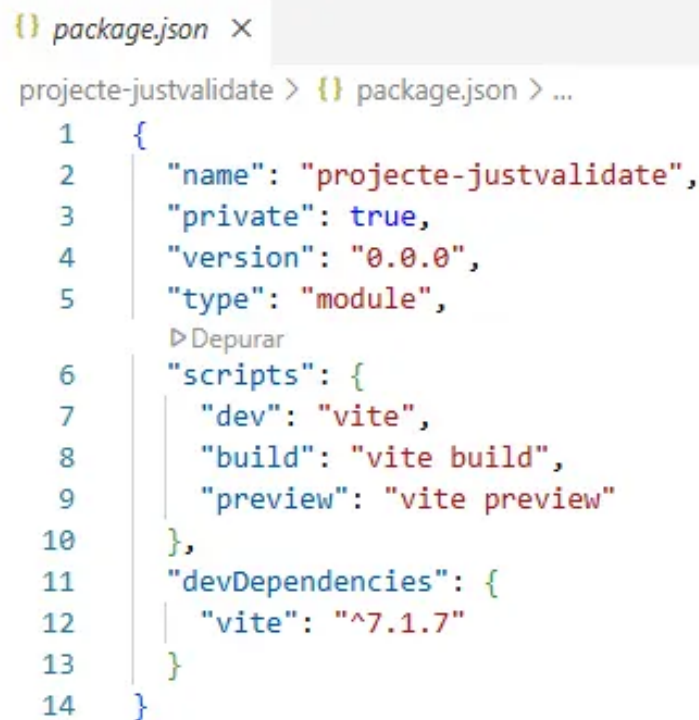
Un cop tinguem el projecte de Vite creat i les dependències instal·lades, hem d'afegir JustValidate al projecte. Dins de la ruta del projecte farem:

```
1 npm install just-validate
```

Al fer l'acció anterior, el que estem fent, a part d'instal·lar l'eina que necessitem, és escriure en el fitxer on hi ha les referències del projecte, `package.json`, que aquest projecte requereix l'eina `just-validate`.

Fixem-nos com teníem l'arxiu `package.json` abans d'instal·lar `just-validate` (figura 5.6):

FIGURA 4.6. fitxer `package.json` abans d'instal·lar "just-validate".



```
{ package.json X
projecte-justvalidate > {} package.json > ...
1  {
2    "name": "projecte-justvalidate",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "preview": "vite preview"
10   },
11   "devDependencies": {
12     "vite": "^7.1.7"
13   }
14 }
```

I tot seguit el mateix arxiu just després d'instal·lar `just-validate` (figura 5.7):

FIGURA 4.7. fitxer package.json després d'instal·lar "just-validate".

```
{} package.json X
projecte-justvalidate > {} package.json > ...
1  {
2    "name": "projecte-justvalidate",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "preview": "vite preview"
10   },
11   "devDependencies": {
12     "vite": "^7.1.7"
13   },
14   "dependencies": {
15     "just-validate": "^4.3.0"
16   }
17 }
18
```

L'arxiu **package.json** és un fitxer essencial en projectes de Node.js que conté informació sobre el projecte, com el nom, la versió i les dependències que necessita per funcionar. Aquestes dependències són altres paquets o llibreries que el projecte utilitza, i estan especificades en el camp `dependencies` o `devDependencies`. Quan executeu el comandament `npm install`, Node Package Manager (NPM) llegeix el `package.json`, descarrega i instal·la totes les dependències esmentades a les versions especificades, assegurant-se que el projecte tingui totes les llibreries necessàries per funcionar correctament.

Creació del formulari i les regles de validació

El formulari de validació, `index.html` per exemple, serà exactament el mateix que en l'exemple anterior. Ara, però, caldrà agafar el punt d'entrada de la lògica que serà el fitxer `main.js`.

Fixem-nos que ara el fitxer `main.js` l'afegim dins de la carpeta `src/` seguint l'estructura que Vite crea per defecte:

```
2 <html lang="ca">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-
      scale=1.0" />
6     <title>Validació de Formulari amb JustValidate</title>
7   </head>
8   <body>
9     <div class="container">
10      <h3>Formulari de Contacte</h3>
11      <form id="formulariContacte">
12        <div class="form-group">
13          <label for="nom">Nom complet:</label>
14          <input type="text" id="nom" name="nom" />
15        </div>
16        <div class="form-group">
17          <label for="email">Correu electrònic:</label>
18          <input type="email" id="email" name="email" />
19        </div>
20        <div class="form-group">
21          <label for="edat">Edat:</label>
22          <input type="number" id="edat" name="edat" />
23        </div>
24        <button type="submit">Enviar</button>
25      </form>
26      <p id="missatge"></p>
27    </div>
28    <script type="module" src="src/main.js"></script>
29  </body>
30 </html>
```

Ara escrivim les regles de validació a main.js:

```
1 import JustValidate from 'just-validate';
2 import "./style.css";
3
4 const validator = new JustValidate('#formulariContacte');
5
6 validator
7   .addField('#nom', [
8     { rule: 'required', errorMessage: 'El nom és obligatori' },
9     { rule: 'minLength', value: 3, errorMessage: 'Mínim 3 carà
      cters' },
10  ])
11   .addField('#email', [
12     { rule: 'required', errorMessage: 'El correu és obligatori'
      },
13     { rule: 'email', errorMessage: 'El correu no és vàlid' },
14  ])
15   .addField('#edat', [
16     { rule: 'required', errorMessage: "L'edat és obligatòria" },
17     { rule: 'number', errorMessage: 'Només es permeten números'
      },
18     { rule: 'minNumber', value: 18, errorMessage: "Has de ser
      major d'edat" },
19  ])
20   .onSuccess(() => {
21     document.getElementById('missatge').textContent = 'Formulari
      correcte!';
22   });
```

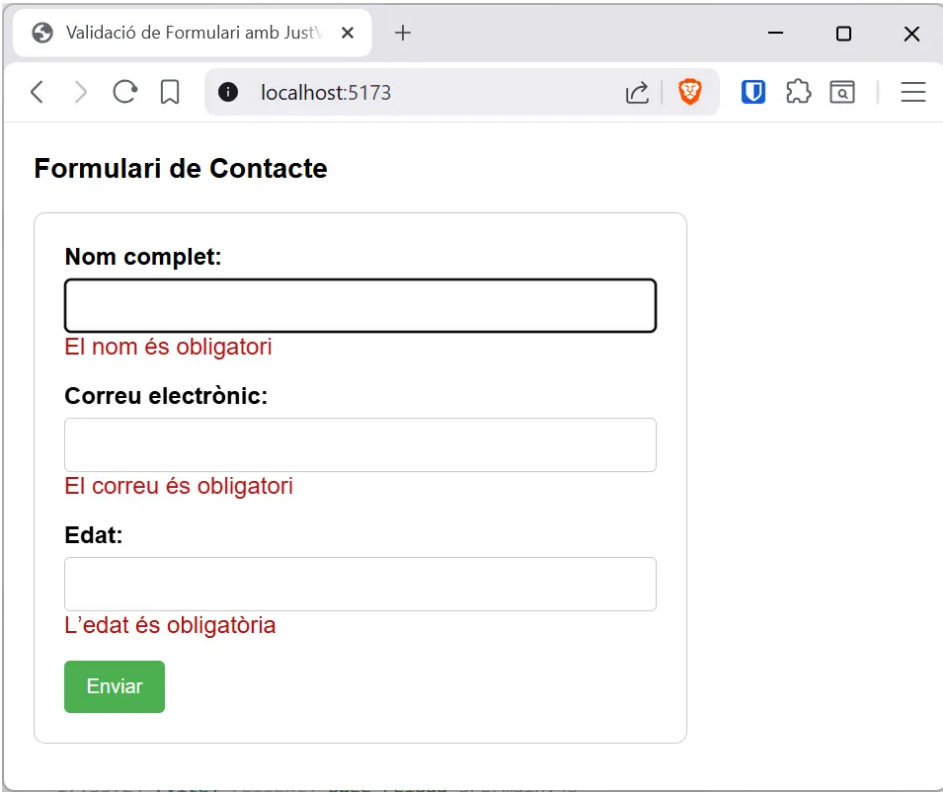
I afegirem els estils a l'arxiu `style.css`:

```
1  body {
2    font-family: Arial, sans-serif;
3    margin: 20px;
4  }
5
6  form {
7    max-width: 400px;
8    padding: 20px;
9    border: 1px solid #ccc;
10   border-radius: 8px;
11 }
12
13 .form-group {
14   margin-bottom: 15px;
15 }
16
17 label {
18   display: block;
19   margin-bottom: 5px;
20   font-weight: bold;
21 }
22
23 input[type='text'],
24 input[type='number'],
25 input[type='email'] {
26   width: 100%;
27   padding: 10px;
28   border: 1px solid #ccc;
29   border-radius: 4px;
30   box-sizing: border-box;
31 }
32
33 button {
34   background-color: #4CAF50;
35   color: white;
36   padding: 10px 15px;
37   border: none;
38   border-radius: 4px;
39   cursor: pointer;
40 }
41
42 .js-validate-error-label {
43   color: red;
44   font-size: 0.9em;
45   margin-top: 5px;
46 }
47
48 .is-invalid {
49   border-color: red;
50 }
```

Execució del projecte

Un cop tinguem tots els fitxers configurats, executarem el servidor de desenvolupament amb **Vite** (figura 4.8):

```
1  npm run dev
```

FIGURA 4.8. Execució del formulari amb validació de camps

The screenshot shows a web browser window with the title "Validació de Formulari amb Just". The address bar displays "localhost:5173". The page content is titled "Formulari de Contacte" and contains three input fields, each with a red validation message below it:

- Nom complet:** The input field is empty. Below it, the message "El nom és obligatori" is displayed in red.
- Correu electrònic:** The input field is empty. Below it, the message "El correu és obligatori" is displayed in red.
- Edat:** The input field is empty. Below it, the message "L'edat és obligatòria" is displayed in red.

At the bottom of the form is a green button labeled "Enviar".

5. Framework Express. Formularis: enviament de dades

En aquesta lliçó ens endinsarem en el món dels *frameworks* per al desenvolupament de *backend* en JavaScript amb ExpressJS (o simplement, Express). Aquest framework ens permetrà crear aplicacions web de manera eficient i senzilla. A més, aprendrem a crear un servidor que sigui capaç de rebre dades des de formularis HTML, i a emmagatzemar-les en una base de dades SQLite.

5.1 Framework Express

ExpressJS és un *framework* que ens permet gestionar rutes, peticions HTTP i facilitar la creació d'aplicacions web i les API. També ens ajuda a estalviar temps a l'hora de construir un servidor, ja que ens ofereix una sintaxi neta i funcional, així com un conjunt de mòduls per a diverses operacions típiques en aplicacions web.

Per què introduïm ExpressJS en un curs de desenvolupament web del costat del client?

Tot i que aquest mòdul se centra en el desenvolupament web **del costat del client**, és essencial que els desenvolupadors entenguem com es comuniquen les aplicacions amb el **costat del servidor**. En el món real, les aplicacions web no treballen de manera aïllada: gairebé sempre necessiten **enviar i rebre dades** a través d'una API o un servidor *backend*.

Aprendre els fonaments d'ExpressJS ens permet:

- **Comprendre el cicle complet d'una aplicació web**, des de la captura de dades amb formularis fins al seu processament i emmagatzematge al servidor.
- **Treballar amb peticions HTTP (GET, POST)** i entendre com s'intercanvia informació entre el navegador i el servidor.
- **Experimentar amb un *backend* lleuger i accessible**, escrit també en JavaScript, que no requereix grans coneixements previs de programació del costat del servidor.
- **Desenvolupar autonomia i visió global** del funcionament d'un projecte web, millorant la seva capacitat per integrar el *frontend* amb serveis reals.
- **Preparar-se per futurs entorns professionals**, on la comunicació entre el client i el servidor és constant i indispensable.

A més, ExpressJS és un *framework* ideal per a introduir aquests conceptes, ja que és **simple, ràpid i compatible** amb els coneixements que els alumnes ja tenen de JavaScript. No es tracta de convertir-nos en desenvolupadors *backend*, sinó de dotar-nos **d'una base pràctica** que ens permeti entendre i provar la part del servidor amb eines properes i actuals.

5.1.1 Què és ExpressJS?

ExpressJS és un *framework* minimalista per a **Node.js** que facilita crear **servidors web i les API**. Proporciona eines senzilles per:

- Definir **rutes** (els URL) i respondre a **peticions HTTP** (GET, POST, PUT, DELETE...).
- Aplicar **middleware** (funcions que processen peticions abans d'arribar a la ruta final).
- Servir **fitxers estàtics** (HTML, CSS, JS, imatges).
- Gestionar **errors** de manera coherent.

Pensem en **ExpressJS** com una capa molt fina a sobre del servidor HTTP de Node que ens dona una sintaxi neta i unes bones pràctiques per construir aplicacions web.

5.1.2 Per a què serveix i com funciona? Alguns conceptes claus

Per a nosaltres, en aquest mòdul, ExpressJS ens servirà sobretot per **rebre dades des de formularis i respondre** al client amb confirmacions o dades.

Però també serveix per més coses, per exemple:

- **API REST** per comunicar el *frontend* amb dades (JSON).
- **Recepció de formularis** i processament de dades.
- **Renderitzat del costat del servidor** (SSR) amb motors de plantilles (EJS, Pug...), si cal.
- **Serveis lleugers**: *proxys*, *webhooks*, passarel·les, microserveis.

Procés de **funcionament** d'ExpressJS:

1. El navegador o el client fa una **petició HTTP** (ex.: GET / o POST /contacte).

2. ExpressJS passa la petició per una **cadena de middleware** (per exemple, per llegir el cos, fer logs, comprovar permisos...).
3. Una **ruta** coincideix amb el camí i el mètode (`app.get('/path', handler)`).
4. El **handler** (funció) construeix i envia una **resposta** (`res.send(...)`, `res.json(...)`, `res.status(...).send(...)`).

Conceptes clau relacionats amb ExpressJS:

- **Ruta:** associació entre un **mètode HTTP** (GET/POST/...) i un **camí** (`/`, `/api/notes`, `/users/:id`).
- **Middleware:** funció (`req`, `res`, `next`) que pot:
 - llegir/modificar `req` o `res`.
 - tallar el flux responent (`res.send(...)`).
 - o continuar amb `next()`.
- **req / res:** objectes de **sol·licitud** i **resposta** de Node/Express.
- **Fitxers estàtics:** carpeta (p. ex. `public/`) que Express serveix automàticament (`app.use(express.static('public'))`).
- **Parsers:** per llegir **JSON** i **formularis** (`express.json()`, `express.urlencoded()`).

ExpressJS ja incorpora **parsers per a JSON i formularis URL-encoded**. En materials antics es feia servir `body-parser`; actualment el més simple és `express.json()` i `express.urlencoded()`.

5.1.3 Exemple pas a pas: Mini servidor amb rutes GET/POST

En aquest exemple, crearem un mini servidor utilitzant ExpressJS que serà capaç de gestionar rutes HTTP de tipus GET i POST. Implementarem una ruta GET per obtenir dades enviades per URL, com ara un paràmetre de consulta, i una ruta POST per rebre dades des de formularis o peticions en format JSON. A més, aprendrem a servir fitxers estàtics, com pàgines HTML, que podrem provar des del navegador. Aquest servidor també inclourà el maneig bàsic d'errors per garantir que les rutes no trobades i les possibles fallades siguin gestionades correctament. A través d'aquest exercici, entendrem com funcionen les comunicacions entre el client i el servidor en una aplicació web.

Objectiu:

- Arrencar un servidor ExpressJS.

- Definir rutes **GET** i **POST**.
- Llegir dades del **query string** i del **cos** (JSON/formulari).
- Servir fitxers estàtics

Servir fitxers estàtics ens servirà per a l'apartat que trobareu més avall: "Formularis: enviament de dades al servidor".

Requisits:

- **Node.js** instal·lat (LTS recomanat).
- Un editor (VS Code, etc.).

Pas 1) Crear projecte

Per començar amb el projecte, el primer pas serà configurar l'entorn de treball. Crearem una nova carpeta per al nostre projecte i inicialitzarem un projecte Node.js utilitzant npm. A continuació, instal·larem Express, que serà el *framework* que utilitzarem per gestionar les rutes i les peticions HTTP. També tindrem l'opció d'instal·lar Nodemon, una eina útil que automàticament reinicia el servidor quan realitzem canvis al codi. Aquest pas ens permetrà tenir la infraestructura bàsica preparada per començar a desenvolupar les funcionalitats del servidor.

```
1 mkdir projecte-express
2 cd projecte-express
3 npm init -y
4 npm install express
```

Opcionalment, podem instal·lar **nodemon** que és una eina que reinicia el servidor quan detecta que s'han fet canvis al codi:

```
1 npm install -D nodemon
```

Al `package.json`, afegirem els *scripts* `dev` i `start`:

```
1 {
2   "name": "projecte-express",
3   "version": "1.0.0",
4   "description": "",
5   "main": "app.js",
6   "scripts": {
7     "test": "echo \"Error: no test specified\" && exit 1",
8     "dev": "nodemon app.js",
9     "start": "node app.js"
10  },
11  "keywords": [],
12  "author": "",
13  "license": "ISC",
14  "dependencies": {
15    "express": "^5.1.0"
16  },
17  "devDependencies": {
18    "nodemon": "^3.1.10"
19  }
20 }
```

Els *scripts* són comandes de **npm** (Node Package Manager) per a diferents tasques relacionades amb el projecte. En aquest cas:

- **“dev”**: **“nodemon app.js”**: Aquest script serveix per executar l'aplicació en mode de desenvolupament, utilitzant **Nodemon**. Nodemon monitora els arxius del projecte i reinicia l'aplicació automàticament cada vegada que detecta un canvi en els arxius (com per exemple un canvi en el codi).
- **“start”**: **“node app.js”**: Aquest script és per executar l'aplicació en producció o en un entorn on no es necessita reinici automàtic com en el cas de Nodemon. Utilitza el comandament **Node.js** per executar el fitxer principal (app.js).

Pas 2) Estructura bàsica de carpetes

Un cop configurat el projecte, organitzarem els fitxers en una estructura senzilla per facilitar el desenvolupament i la gestió del codi. La carpeta principal del projecte contindrà el fitxer app.js, que serà el punt d'entrada de l'aplicació, i una carpeta public/ on guardarem els fitxers estàtics com HTML, CSS i altres recursos que el servidor servirà als usuaris. Aquesta estructura ens ajudarà a mantenir el codi organitzat i permetrà una fàcil acció en el desenvolupament i les proves del projecte:

```
1 projecte-express/  
2 | app.js  
3 |-- public/  
4 |   |-- index.html  
5 |   |-- style.css
```

Crearem un public/index.html simple (ens servirà per provar-ho des del navegador):

```
1 <!DOCTYPE html>  
2 <html lang="ca">  
3   <head>  
4     <meta charset="utf-8" />  
5     <title>Projecte Express</title>  
6   </head>  
7   <body>  
8     <h1>Hola des d'Express!</h1>  
9     <p>  
10       Prova <em>/api/hola?nom=Maria</em> o envia un POST a  
11       <em>/api/eco</em>.  
12     </p>  
13   </body>  
14 </html>
```

Pas 3) Codi del servidor: app.js

Ara que tenim la infraestructura bàsica preparada, crearem el fitxer app.js, que serà el cor del nostre servidor Express. En aquest fitxer, definirem les rutes i

configurarem els diferents elements necessaris per al funcionament del servidor, com els *middleware* per processar les peticions, la gestió de fitxers estàtics i les respostes de les rutes. També inclourem les rutes de demostració, com la ruta GET per obtenir un missatge personalitzat i la ruta POST per retornar les dades rebudes. Aquest codi serà la base sobre la qual construïrem la funcionalitat del servidor:

```
1  const express = require("express");
2
3  const app = express();
4  const PORT = 3000;
5
6  // 1) Middleware de parsers
7  app.use(express.json()); // per cosos JSON (
   application/json)
8  app.use(express.urlencoded({ extended: true })); // per
   formularis HTML (application/x-www-form-urlencoded)
9
10 // 2) Servir fitxers estàtics (public/)
11 app.use(express.static('public'));
12
13 // 3) Rutes de demostració
14
15 // GET /api/hola?nom=Maria -> { missatge: "Hola, Maria!" }
16 app.get('/api/hola', (req, res) => {
17   const nom = req.query.nom || 'món';
18   res.json({ missatge: 'Hola, ${nom}!' });
19 });
20
21 // POST /api/eco (rep JSON o formulari, i el retorna)
22 app.post('/api/eco', (req, res) => {
23   // req.body conté les dades del client (JSON o formulari)
24   res.status(201).json({
25     rebut: true,
26     dades: req.body
27   });
28 });
29
30 // 4) Ruta 404 per a camins no existents d'API
31 app.use('/api', (req, res) => {
32   res.status(404).json({ error: 'Ruta d'API no trobada' });
33 });
34
35 // 5) Middleware de gestió d'errors (exemple bàsic)
36 app.use((err, req, res, next) => {
37   console.error('Error:', err);
38   res.status(500).json({ error: 'Error intern del servidor' });
39 });
40
41 // 6) Posar el servidor en marxa
42 app.listen(PORT, () => {
43   console.log('Servidor escoltant a http://localhost:${PORT}');
44 });
```

Pas 4) Executar el projecte

Un cop tinguem el codi del servidor creat, el següent pas serà executar-lo i posar-lo en marxa. Per fer-ho, utilitzarem l'script de desenvolupament amb Nodemon, que reiniciarà automàticament el servidor cada cop que fem canvis en el codi. Un

cop el servidor estigui en marxa, podrem obrir el navegador i provar les rutes definides, com la pàgina estàtica inicial i les API que hem creat, comprovant així que tot funcioni correctament. Aquest procés ens permetrà veure els resultats en temps real i verificar que el servidor estigui responnent a les peticions com esperem:

```
1 npm run dev
```

Obrirem el navegador a: <http://localhost:3000>

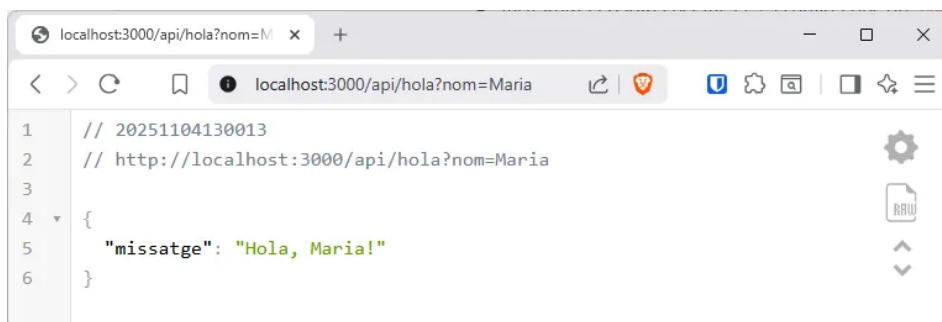
- Visitarem la pàgina estàtica (/) servida des de public/ (figura 5.1).

FIGURA 5.1. Execució del projecte



- Provarem l'API:
 - **GET** (en la barra del navegador): <http://localhost:3000/api/hola?nom=Maria> (figura 5.2)

FIGURA 5.2. Petició API amb Express (mètode GET)



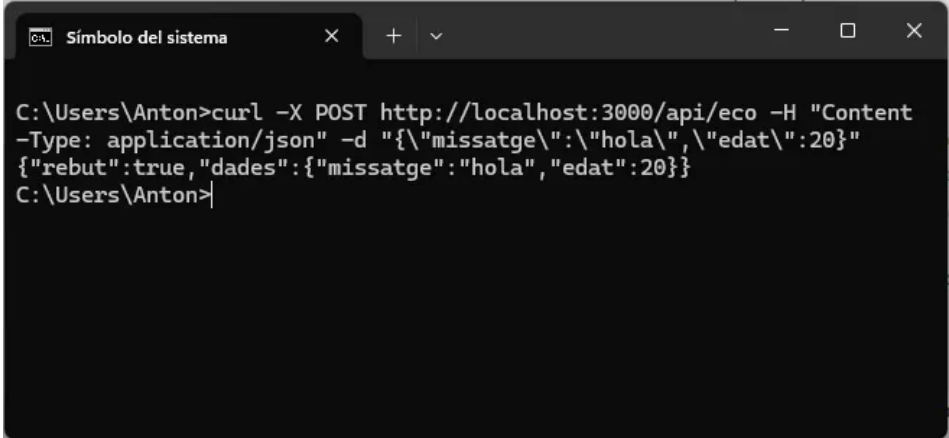
- **POST** (des de la consola amb cURL, o des de l'aplicació Postman que s'ha d'instal·lar) (figura 5.3):

```
1 curl -X POST http://localhost:3000/api/eco -H "Content-Type: application/json" -d '{"missatge":"hola","edat":20}'
```

Resposta esperada (figura 5.3):

```
1 {  
2     "rebut": true,  
3     "dades": { "missatge": "hola", "edat": 20 }  
4 }
```

FIGURA 5.3. Resposta rebuda de l'API



```
C:\Users\Anton>curl -X POST http://localhost:3000/api/eco -H "Content  
-Type: application/json" -d "{\"missatge\":\"hola\", \"edat\":20}"  
{\"rebut\":true,\"dades\":{\"missatge\":\"hola\", \"edat\":20}}  
C:\Users\Anton>
```

cURL és una eina de línia d'ordres que permet realitzar peticions HTTP a un servidor, ideal per provar les API i serveis web. Es pot utilitzar per enviar peticions GET, POST, PUT, DELETE, entre altres, i permet especificar capçaleres, cos de la petició i altres paràmetres.

Postman és una aplicació gràfica que facilita el testatge de les API. Ofereix una interfície d'usuari més amigable, on pots crear peticions HTTP de manera visual, gestionar entorns, veure les respostes en diferents formats i automatitzar tests per validar el comportament de l'API. Ambdues eines són molt útils per desenvolupar i depurar serveis web.

5.2 Formularis: enviament de dades al servidor

Els **formularis HTML** són una eina fonamental per a capturar dades dels usuaris. En aquesta secció, veurem com enviar aquestes dades al servidor utilitzant el mètode POST i com ExpressJS pot gestionar aquesta informació. Per facilitar el procés, utilitzarem la llibreria **body-parser** per analitzar les dades de les peticions POST.

Quan l'usuari envia un formulari, el navegador crea una **petició HTTP** (normalment de tipus POST) i envia les dades al servidor.

El **servidor Express** rep aquestes dades, les processa i pot:

- Validar-les
- Desar-les a una base de dades (o a un fitxer, o qualsevol altra cosa)

- Retornar una resposta al client

Aquesta secció ens permet **tancar el cicle complet**: *Frontend* (formulari) → Servidor (Express) → Emmagatzematge (SQLite) → Resposta al client.

5.2.1 Teoria bàsica: com funciona l'enviament de dades

Quan enviem un formulari com aquest...

```
1 <form action="/formulari" method="POST">
2   <label>Nom: <input type="text" name="nom" /></label>
3   <label>Correu: <input type="email" name="correu" /></label>
4   <button type="submit">Enviar</button>
5 </form>
```

..., el navegador fa una petició:

```
1 POST /formulari HTTP/1.1
2 Content-Type: application/x-www-form-urlencoded
3 Body: nom=Anna&correu=anna%40mail.com
```

A Express podem llegir aquest contingut amb:

```
1 app.use(express.urlencoded({ extended: true }));
```

Si l'enviament fos en **JSON**, faríem servir:

```
1 app.use(express.json());
```

5.2.2 Exemple pràctic: Formulari que envia dades al servidor

En aquest exemple pràctic, crearem un formulari HTML simple que permetrà als usuaris enviar dades al servidor mitjançant el mètode POST. Aquest formulari recollirà informació com el nom i el correu electrònic, i un cop enviades, les dades seran processades pel servidor Express. El servidor les emmagatzemarà a una base de dades SQLite i retornarà una resposta al client per confirmar que les dades s'han rebut correctament. A través d'aquest exercici, veurem com gestionar dades de formularis en una aplicació web i com interactuar amb una base de dades des del servidor.

Estructura del projecte

Crearem la següent estructura per desenvolupar aquest exemple:

```
1 formulari-express/  
2 | app.js  
3 | dades.db ← base de dades SQLite  
4 |-- public/  
5   |-- index.html  
6   |-- style.css
```

Instal·lacions necessàries:

```
1 mkdir formulari-express  
2 cd formulari-express  
3 npm init -y  
4 npm install express sqlite3
```

(Opcional: `npm install -D nodemon` i afegir l'script "dev": "nodemon app.js")

Codi del servidor (app.js)

En aquest apartat, escriurem el codi del servidor en el fitxer `app.js`, on configurarem Express per gestionar les peticions HTTP.

Primer, definirem els *middleware* necessaris per processar les dades del formulari i del JSON, així com per servir els fitxers estàtics. Després, establim la connexió a la base de dades SQLite i crearem una taula per emmagatzemar les dades del formulari. Finalment, implementarem una ruta POST per rebre les dades del formulari, validar-les i desar-les a la base de dades, així com una ruta GET per obtenir els contactes desats.

Aquest codi serà el nucli del nostre servidor i gestionarà la comunicació entre el client i la base de dades:

```
1 const express = require("express");  
2 const sqlite3 = require("sqlite3");  
3  
4 const app = express();  
5 const PORT = 3000;  
6  
7 // Middleware per llegir formularis i JSON  
8 app.use(express.urlencoded({ extended: true }));  
9 app.use(express.json());  
10  
11 // Servir fitxers estàtics  
12 app.use(express.static('public'));  
13  
14 // Connexió a la base de dades SQLite  
15 const db = new sqlite3.Database("./dades.db", (err) => {  
16   if (err) console.error(err.message);  
17   else console.log("Connexió a SQLite establerta.");  
18 });  
19  
20 // Crear la taula de contactes si no existeix  
21 db.run(  
22   'CREATE TABLE IF NOT EXISTS contactes (  
23     id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
24     nom TEXT NOT NULL,  
25     correu TEXT NOT NULL  
26   )'  
27 );  
28  
29 // Ruta POST per rebre les dades del formulari  
30 app.post('/formulari', async (req, res) => {  
31   const { nom, correu } = req.body;  
32  
33   if (!nom || !correu) {  
34     return res.status(400).send('Falten camps obligatoris');  
35   }  
36  
37   db.run('INSERT INTO contactes (nom, correu) VALUES (?, ?)', [  
38     nom, correu]);  
39  
40   res.send('<h2>Dades rebudes correctament</h2>  
41     <p>Nom: ${nom}</p>  
42     <p>Correu: ${correu}</p>  
43     <a href="/">Tornar</a>');  
44  
45   });  
46  
47 // Obtener tots els contactes (GET /contactes)  
48 app.get("/contactes", (req, res) => {  
49   db.all("SELECT * FROM contactes", [], (err, rows) => {  
50     if (err) return res.status(500).json({ error: err.  
51       message });  
52     res.json(rows);  
53   });  
54 });  
55  
56 // Posar en marxa el servidor  
57 app.listen(PORT, () => {  
58   console.log('Servidor actiu a http://localhost:${PORT}');  
59 });
```

Formulari HTML (public/index.html)

Ara crearem el formulari HTML que els usuaris utilitzaran per enviar les seves dades al servidor. El formulari permetrà introduir el nom i el correu electrònic, que seran enviats mitjançant una petició POST al servidor Express. Inclourem les etiquetes bàsiques de formulari, com input per als camps de text i button per enviar la informació.

Aquest formulari es presentarà en una pàgina web senzilla i utilitzarà l'estil bàsic definit en un fitxer CSS per millorar la seva visualització.

Quan els usuaris enviïn el formulari, les dades seran enviades al servidor per ser processades i emmagatzemades:

```
1 <!DOCTYPE html>  
2 <html lang="ca">  
3 <head>  
4   <meta charset="utf-8" />  
5   <title>Formulari Express</title>  
6   <link rel="stylesheet" href="style.css">  
7 </head>  
8 <body>
```

```
9    <h1>Enviament de dades al servidor</h1>
10
11    <form action="/formulari" method="POST">
12        <label>Nom:<br>
13            <input type="text" name="nom" required>
14        </label><br><br>
15
16        <label>Correu:<br>
17            <input type="email" name="correu" required>
18        </label><br><br>
19
20        <button type="submit">Enviar</button>
21    </form>
22
23    <hr>
24
25    <p><a href="/contactes">Veure tots els contactes (JSON)</a></p>
26</body>
27</html>
```

CSS bàsic (public/style.css)

Afegirem uns estils bàsics per millorar l'experiència de l'usuari:

```
1  body {
2      font-family: sans-serif;
3      max-width: 600px;
4      margin: 40px auto;
5      line-height: 1.6;
6  }
7
8  label {
9      display: block;
10 }
11
12 input, button {
13     padding: 6px;
14     font-size: 1rem;
15 }
16
17 button {
18     background: #007bff;
19     color: white;
20     border: none;
21     cursor: pointer;
22     border-radius: 4px;
23 }
24
25 button:hover {
26     background: #0056b3;
27 }
```

Proves

Tot seguit realitzarem les proves per verificar que el servidor funciona correctament. Un cop tinguem el codi del servidor i el formulari configurats, provarem la

funcionalitat enviant dades des del formulari HTML al servidor i comprovarem que aquestes dades es desin correctament a la base de dades. Utilitzarem el navegador per omplir el formulari i enviar les dades, i després accedirem a les rutes definides per veure les dades desades. També comprovarem les respostes de les peticions GET i POST per assegurar-nos que el sistema estigui processant la informació com es preveu.

Pas 1) Executarem:

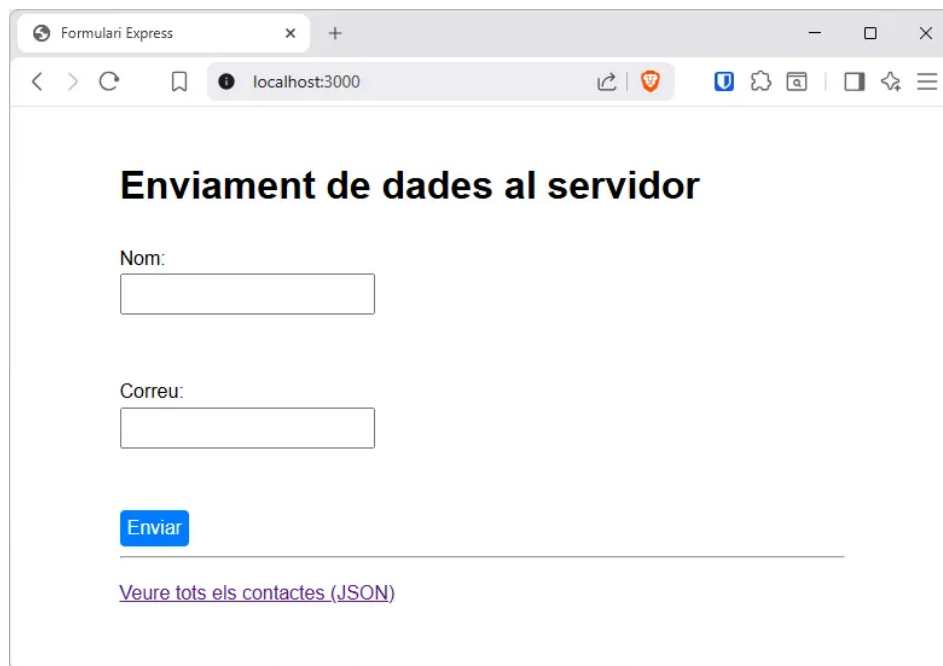
```
1 node app.js
```

O, si hem creat l'script dev, farem:

```
1 npm run dev
```

Pas 2) Obrim <http://localhost:3000> (figura 5.4).

FIGURA 5.4. Formulari



The screenshot shows a web browser window with the title 'Formulari Express'. The address bar shows 'localhost:3000'. The main content area has the heading 'Enviament de dades al servidor'. Below the heading are two input fields: 'Nom:' and 'Correu:'. Below the 'Correu:' field is a blue button labeled 'Enviar'. At the bottom of the form is a link that says 'Veure tots els contactes (JSON)'.

Pas 3) Omplim dades i enviem el formulari (figura 5.5).

FIGURA 5.5. Omplim les dades del formulari

Formulari Express

localhost:3000

Enviament de dades al servidor

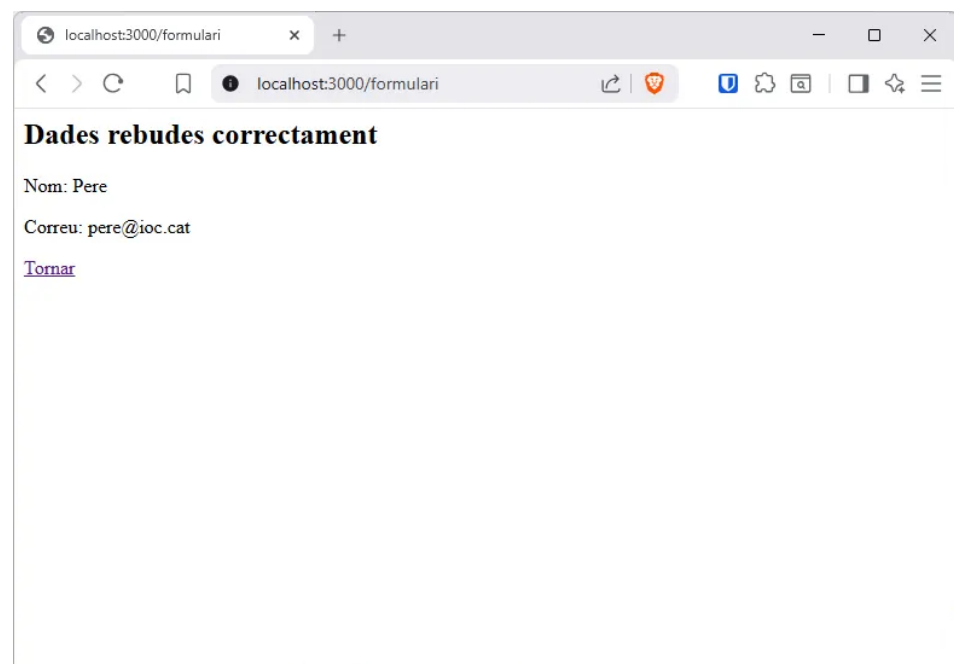
Nom:

Correu:

[Enviar](#)

[Veure tots els contactes \(JSON\)](#)

Pas 4) Express mostrarà una pàgina amb el missatge de confirmació (figura 5.6).

FIGURA 5.6. Resposta del servidor

localhost:3000/formulari

localhost:3000/formulari

Dades rebudes correctament

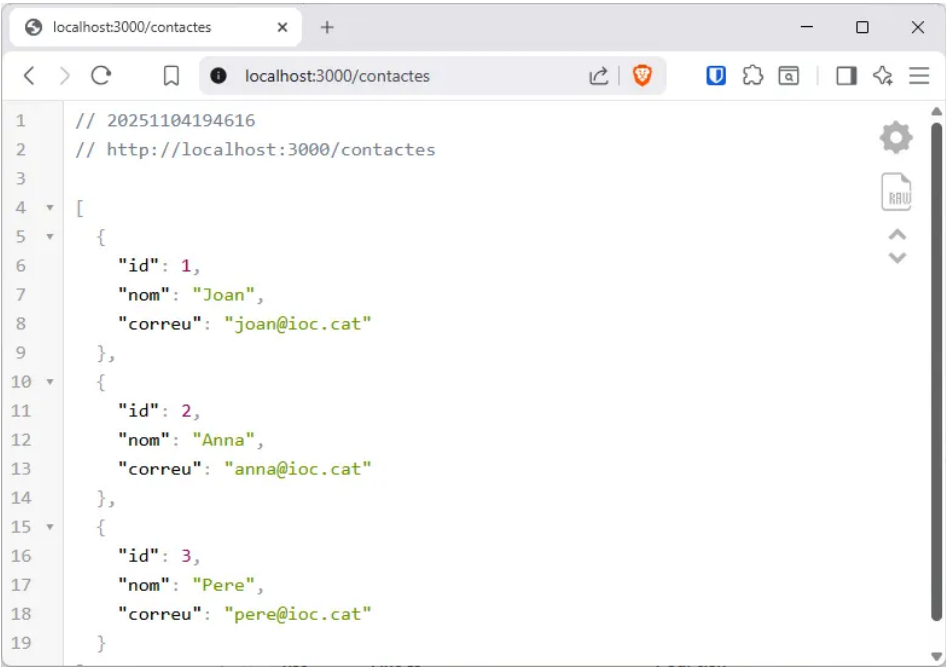
Nom: Pere

Correu: pere@ioc.cat

[Tornar](#)

Pas 5) Provarem <http://localhost:3000/contactes> per veure les dades desades a la base SQLite (figura 5.7).

FIGURA 5.7. Informació provinent de la base de dades



Explicació pas a pas

Si hem provat la nostra aplicació i hem enviat les dades a través del formulari, la taula 5.1 mostra el que internament ocorre:

TAULA 5.1. Passos comunicació formulari amb servidor

Pas	Què fa?	Codi clau
1	El navegador envia una petició POST amb les dades del formulari.	<form action="/formulari" method="POST">
2	Express intercepta la petició i llegeix el cos.	app.use(express.urlencoded(...))
3	El servidor processa les dades i les desa.	db.run('INSERT INTO contactes...')
4	Express retorna una resposta HTML al client.	res.send(...)
5	L'usuari veu un missatge de confirmació i pot consultar les dades desades.	/contactes